

Price Sync — Hướng dẫn cài đặt & vận hành

Cài đặt, chạy và kiểm thử hệ thống đồng bộ giá từ HQ xuống POS (Spring Boot + React + PostgreSQL).

Spring Boot 3.5 · Java 17

React 19 · Vite · TypeScript

PostgreSQL 16 · Flyway

Docker Compose

Repo: XuanSang2005/Price_Sync

Phiên bản tài liệu: 1.0 · Ngày 23/07/2026 · App version 2.4.0 (UAT)

Mục lục

1. Hệ thống này là gì
2. Yêu cầu môi trường (Prerequisites)
3. Lấy mã nguồn
4. Biến môi trường & secrets
5. Cách A — Chạy toàn bộ bằng Docker Compose (khuyến nghị)
6. Cách B — Chạy local để phát triển
7. Đóng gói bản production (1 cổng duy nhất)
8. Kiểm tra sau cài đặt (Smoke test)
9. Cấu hình sau cài đặt (không cần redeploy)
10. Xử lý sự cố thường gặp
11. Phụ lục — cổng, endpoint, migration

1. Hệ thống này là gì

Price Sync là middleware đồng bộ giá HQ → POS cho Oracle Retail Xstore. Mỗi đêm, HQ gửi (POST) một lô giá dạng JSON tới hệ thống; hệ thống xác thực, kiểm tra, ánh xạ (mapping) và tạo ra file **MNT** (Oracle DataLoader) rồi ghi vào thư mục Xcenter inbound để Xstore phân phối xuống POS.

Về mặt triển khai, đây là một **modular monolith** gồm 2 phần đóng gói chung:

- **Backend** (`backend/`) — ứng dụng Spring Boot: API intake + worker xử lý nền + ops console API. PostgreSQL giữ toàn bộ trạng thái; Flyway tự tạo & nâng cấp schema khi khởi động.
- **Frontend** (`frontend/`) — SPA React (Vite + TypeScript + TanStack Router + Tailwind). Khi đóng gói production, UI được build thẳng vào `static/` của Spring nên chỉ chạy **một cổng 8080**.

Luồng xử lý: `POST /api/v1/price-events` (202) → lưu batch `RECEIVED` → worker định kỳ claim (`SELECT ... FOR UPDATE SKIP LOCKED`) → validate → map → build file MNT → ghi ra Xcenter inbound → batch `WRITTEN` (hoặc `PARTIAL` / `FAILED` , có retry & email alert).

Chọn cách cài nào? Muốn chạy thử/demo nhanh nhất → dùng **Cách A (Docker Compose)**: một lệnh dựng cả Postgres + app + hộp thư mail. Muốn code/sửa & hot-reload → dùng **Cách B (local dev)**.

2. Yêu cầu môi trường (Prerequisites)

2.1. Cho Cách A — Docker A

Công cụ	Phiên bản tối thiểu	Ghi chú
Docker Engine	24+ (đã kiểm với 29)	Có kèm <code>docker compose</code> v2/v5
Docker Compose	v2+ (đã kiểm với v5)	Dùng lệnh <code>docker compose</code> (không phải <code>docker-compose</code>)
Git	bất kỳ	Để clone mã nguồn

Không cần cài Java / Node / Postgres trên máy — mọi thứ nằm trong image.

2.2. Cho Cách B — Local dev B

Công cụ	Phiên bản	Ghi chú
JDK	17 (bắt buộc)	Ví dụ Temurin 17. <code>pom.xml</code> đặt <code>java.version=17</code>
Maven	—	Không cần cài — dùng Maven Wrapper <code>./mvnw</code> có sẵn trong <code>backend/</code>
Node.js	20+ (khuyến nghị 22)	Docker build dùng <code>node:22</code> . Frontend cần npm
PostgreSQL	16	Chạy local ở cổng 5432, có database <code>price_sync</code>
Git	bất kỳ	

Kiểm tra nhanh môi trường (Cách B)

```
# mỗi lệnh phải in ra phiên bản, không báo "command not found"
java -version      # cần openjdk 17.x
node -v            # cần v20+ (v22 lý tưởng)
psql --version     # cần PostgreSQL 16.x
git --version
```

3. Lấy mã nguồn

```
git clone https://github.com/XuanSang2005/Price_Sync.git
cd Price_Sync
```

Cấu trúc thư mục chính:

```
Price_Sync/
├── Dockerfile                # build 3 tầng: UI → jar (kèm UI) → runtime JRE
├── docker-compose.yml       # db (Postgres) + app + mailpit
├── backend/                 # Spring Boot (chạy ./mvnw ở ĐÂY)
│   ├── pom.xml
│   ├── mvnw, mvnw.cmd       # Maven Wrapper – không cần cài Maven
│   └── src/main/resources/
│       ├── application.properties
│       └── db/migration/    # V1..V19 – Flyway tự chạy khi khởi động
└── frontend/               # React + Vite (chạy npm ở ĐÂY)
    ├── package.json
    └── vite.config.ts
```

Lưu ý về file bị gitignore: `run.sh` (script chạy local có chứa mật khẩu Gmail cá nhân) và thư mục `backend/src/main/resources/static/` (UI build sẵn) **không** nằm trong repo. Bạn sẽ tự tạo/khôi phục chúng theo hướng dẫn bên dưới.

4. Biến môi trường & secrets

Từ bản 07/2026, **secret không nằm trong repo** — `application.properties` chỉ chứa placeholder `$ {...}`. Thiếu biến bắt buộc → app **cố tình chết ngay khi khởi động** (fail-fast) với thông báo `Could not resolve placeholder 'HQ_API_KEY'`.

Biến bắt buộc

Biến	Ý nghĩa	Giá trị dùng cho dev/demo
HQ_API_KEY	API key HQ gửi ở header <code>X-Api-Key</code>	<code>hq-secret-key-123</code>
HQ_HMAC_SECRET	Khoá bí mật ký HMAC-SHA256 request	<code>shared-secret-abc</code>

Biến tùy chọn (có mặc định)

Biến	Mặc định	Ý nghĩa
SPRING_DATASOURCE_URL	<code>jdbc:postgresql://localhost:5432/price_sync</code>	Chuỗi kết nối Postgres
SPRING_DATASOURCE_USERNAME	<code>macbookpro</code>	User Postgres (đổi theo máy bạn — xem §6.1)
SPRING_DATASOURCE_PASSWORD	<code>(rỗng)</code>	Mật khẩu Postgres
MAIL_HOST / MAIL_PORT	<code>127.0.0.1 / 1025</code>	SMTP gửi alert khi batch FAILED (mặc định trở Mailpit)
MAIL_AUTH / MAIL_STARTTLS	<code>false / false</code>	Bật khi dùng SMTP thật (vd Gmail)
MAIL_USERNAME / MAIL_PASSWORD	<code>(rỗng)</code>	Thông tin đăng nhập SMTP thật

Tuyệt đối không commit secret. Nếu tạo file `run.sh` để chạy nhanh, nó đã được gitignore sẵn — **đừng** gỡ gitignore. Với SMTP Gmail, dùng **App Password 16 ký tự** (không dùng mật khẩu tài khoản), và đặt qua biến môi trường chứ không viết cứng vào file theo dõi.

5. Cách A — Chạy toàn bộ bằng Docker Compose khuyến nghị

Một lệnh dựng cả **Postgres + app (đã kèm UI) + Mailpit**. Postgres trong container là **mới tinh** → Flyway chạy V1...V19 tạo schema và seed config/mapping sạch, không lẫn dữ liệu test cũ.

5.1. Dựng & chạy

```
# tại thư mục gốc Price_Sync/  
docker compose up --build
```

Lần đầu sẽ mất vài phút (build UI → đóng jar → tải image Postgres/Mailpit). Khi thấy log Spring Boot **Started PriceSyncApplication** là đã sẵn sàng. Secret HQ, datasource và MAIL_HOST đã được khai sẵn trong **docker-compose.yml** — không cần thao tác thêm.

5.2. Các địa chỉ truy cập

Địa chỉ	Nội dung
http://localhost:8080	UI console + API (cùng một cổng)
http://localhost:8080/api/v1/events	API danh sách batch (JSON)
http://localhost:8025	Mailpit — hộp thư web xem email alert khi batch FAILED

Postgres cổng 5432 KHÔNG mở ra host — cố ý, để không đụng Postgres local của bạn. Muốn xem DB trong container: **docker compose exec db psql -U price_sync -d price_sync**.

5.3. Xem file MNT sinh ra

docker-compose.yml mount **./xcenter-inbound** (trên host) vào container, nên file MNT do app ghi ra xuất hiện ngay trên máy bạn:

```
ls -lt xcenter-inbound/  
cat xcenter-inbound/pricesync_*.mnt
```

5.4. Dừng / dọn

```
docker compose down          # dừng, GIỮ dữ liệu DB (volume pgdata)  
docker compose down -v      # dừng và XOÁ sạch dữ liệu DB (chạy lại như mới)
```

Gotcha đã biết khi POST qua Docker: app trong container thấy IP nguồn là gateway của Docker Desktop (**192.168.65.1**), không phải **127.0.0.1** → bị chặn **403** bởi IP allowlist. Trước khi chạy smoke test end-to-end trên Docker, hãy thêm IP đó vào allowlist (xem §8.4 và §9.2). Các endpoint **đọc** (**/api/v1/events**, **/config**, ...) không bị lọc nên vẫn gọi được ngay.

6. Cách B — Chạy local để phát triển dev

Backend và frontend chạy **2 tiến trình riêng**: Spring ở `:8080`, Vite dev server ở `:5173` (hot-reload). Vite proxy mọi request `/api` sang `:8080` nên trình duyệt không dính CORS.

6.1. Chuẩn bị PostgreSQL

Cần một database tên `price_sync` mà app kết nối được. Với Postgres cài bằng Homebrew trên macOS:

```
# khởi động Postgres (nếu chưa chạy)
brew services start postgresql@16

# tạo database — Homebrew tạo sẵn 1 role trùng tên đăng nhập máy bạn
createdb price_sync

# kiểm tra kết nối được
psql -d price_sync -c "select 1;"
```

Bắt buộc đọc — user DB mặc định là giá trị **GHI CỨNG**. `application.properties` đặt `spring.datasource.username=macbookpro` (tên máy của tác giả), mật khẩu rỗng — **không** tự lấy theo tên máy bạn. Nếu login của bạn khác `macbookpro` mà để nguyên, app sẽ chết với `FATAL: role "macbookpro" does not exist`. Vì vậy **luôn override** `SPRING_DATASOURCE_USERNAME` sang role Postgres của bạn khi chạy (lệnh §6.2 dưới đây đã dùng `$(whoami)` để tự khớp — với Homebrew Postgres, role mặc định trùng tên đăng nhập máy bạn).

Không cần tạo bảng tay — **Flyway** sẽ tự tạo mọi bảng và seed dữ liệu (config, mapping_rule) ngay lần chạy đầu.

6.2. Chạy backend

Chạy từ thư mục `backend/`, kèm 2 secret bắt buộc. Mail để mặc định (trở 127.0.0.1:1025 — nếu không có SMTP nào ở đó, gửi alert sẽ chỉ log lỗi và **không** làm gãy pipeline):

```
cd backend
HQ_API_KEY=hq-secret-key-123 \
HQ_HMAC_SECRET=shared-secret-abc \
SPRING_DATASOURCE_USERNAME=$(whoami) \
./mvnw spring-boot:run
```

`SPRING_DATASOURCE_USERNAME=$(whoami)` khớp với role Postgres mặc định của Homebrew (xem cảnh báo §6.1). Nếu role/mật khẩu của bạn khác, thay `$(whoami)` bằng role thật và thêm `SPRING_DATASOURCE_PASSWORD=<mật khẩu>`. Thành công khi thấy `Tomcat started on port 8080` và `Started PriceSyncApplication`. Lúc này API đã sống ở `http://localhost:8080` (nhưng chưa có UI — UI chạy ở tiến trình Vite bên dưới).

6.3. Chạy frontend (tiến trình thứ 2, terminal khác)

```
cd frontend
npm install      # chỉ lần đầu (hoặc khi đổi dependency)
npm run dev      # mở http://localhost:5173
```

Mở <http://localhost:5173>. Mọi lời gọi `/api/...` từ UI được Vite proxy sang backend `:8080`.

Tóm tắt Cách B: Postgres chạy → `backend/`: `./mvnw spring-boot:run` (kèm 2 env) → `frontend/`: `npm run dev` → dùng UI ở `:5173`. Sửa code React nóng ngay; sửa code Java cần khởi động lại backend.

7. Đóng gói bản production (1 cổng duy nhất)

Ở production không chạy Vite. UI được build ra file tĩnh và nhét vào Spring; jar tự phục vụ cả UI lẫn API trên **cổng 8080**. Deep-link (vd `/dashboard`, `/events/12`) và refresh không bị 404 nhờ `SpaController` forward về `index.html`.

7.1. Build thủ công (không Docker)

```
# 1) build UI → đổ vào backend/src/main/resources/static/ (đã cấu hình trong vite.config.ts)
cd frontend
npm install
npm run build

# 2) đóng jar (UI đã nằm trong static/ nên jar tự chứa UI)
cd ../backend
./mvnw clean package -DskipTests

# 3) chạy jar (SPRING_DATASOURCE_USERNAME: xem cảnh báo §6.1)
HQ_API_KEY=hq-secret-key-123 \
HQ_HMAC_SECRET=shared-secret-abc \
SPRING_DATASOURCE_USERNAME=$(whoami) \
java -jar target/price_sync-0.0.1-SNAPSHOT.jar
```

Mở <http://localhost:8080> — giờ có cả UI lẫn API cùng cổng.

Bỏ `-DskipTests` nếu muốn chạy test khi đóng gói — nhưng test cần Postgres và biến môi trường, nên thường tách sang CI.

7.2. Build bằng Docker (khuyến nghị cho production)

`Dockerfile` đã gộp cả 3 bước trên vào một image nhiều tầng (node → maven → JRE). Chỉ cần `docker compose up --build` như §5 — không phải build tay.

8. Kiểm tra sau cài đặt (Smoke test)

8.1. Mở giao diện

Docker/production → <http://localhost:8080>; local dev → <http://localhost:5173>. Thấy trang Dashboard/Events là UI đã sống.

8.2. Gọi các API đọc (không cần xác thực)

Những endpoint này **không** bị lọc bảo mật (chỉ `/api/v1/price-events` mới bị), rất tiện để kiểm tra nhanh app + DB:

```
curl http://localhost:8080/api/v1/events          # danh sách batch (JSON, có thể là [])
curl http://localhost:8080/api/v1/events/metrics # đếm batch theo trạng thái
curl http://localhost:8080/api/v1/config          # các dòng config đã seed
curl http://localhost:8080/api/v1/mappings        # các luật mapping đã seed
```

`/config` phải trả về các key như `abort_threshold`, `ip_allowlist`, `xcenter_inbound_path`... → chứng tỏ Flyway đã seed thành công.

8.3. Gửi thử một batch có ký HMAC (end-to-end)

Endpoint intake yêu cầu 4 lớp: IP allowlist, HMAC-SHA256, API key, và cửa sổ thời gian (chống replay).

Chữ ký = `HMAC-SHA256(secret, X-Timestamp + body)` ở dạng hex. Script bash dưới đây tự ký bằng `openssl`:

```
API_KEY="hq-secret-key-123"
HMAC_SECRET="shared-secret-abc"
TS=$(date +%s)                # epoch giây, phải trong ±300s
BATCH_ID="smoke-$TS"

# body 1 dòng – PHẢI khớp từng byte giữa lúc ký và lúc gửi
BODY="{\"batch_id\": \"${BATCH_ID}\", \"version\": 1, \"generated_at\": \"2026-07-23T09:00:00+07:00\", \"records\": [{\"change_id\": \"${BATCH_ID}-r1\", \"version\": 1, \"item_id\": \"SKU1\", \"store_id_or_zone\": \"STORE_001\", \"price\": 50000, \"currency\": \"VND\", \"effective_start\": \"2026-07-24\", \"effective_end\": null, \"change_type\": \"new\"}]}"

# ký ts+body; awk lấy trường cuối = chuỗi hex (bền với mọi kiểu output openssl)
SIG=$(printf '%s' "$TS$BODY" | openssl dgst -sha256 -hmac "$HMAC_SECRET" | awk '{print $NF}')

curl -i -X POST http://localhost:8080/api/v1/price-events \
  -H "Content-Type: application/json" \
  -H "X-Api-Key: $API_KEY" \
  -H "X-Timestamp: $TS" \
  -H "X-Signature: $SIG" \
  --data "$BODY"
```

Kỳ vọng: `HTTP/1.1 202` kèm JSON xác nhận. Sau tối đa ~10 giây (worker poll mỗi 10s), batch được xử lý và một file MNT xuất hiện trong thư mục Xcenter inbound:

```
# Docker / chạy jar ở gốc: file ở ./xcenter-inbound/  
# chạy ./mvnw ở backend/: file ở backend/xcenter-inbound/  
ls -lt xcenter-inbound/ 2>/dev/null || ls -lt backend/xcenter-inbound/  
# nội dung: FHEAD,2026-07-23 / FDETL,SKU1,S,001,50000,VND,2026-07-24, / FTAIL,1
```

8.4. Nếu chạy trên Docker → thêm IP trước

Trên Docker, POST từ host bị **403** do IP **192.168.65.1** chưa có trong allowlist. Chạy lệnh này một lần (áp dụng tức thì, không cần khởi động lại), rồi lặp lại §8.3:

```
curl -X PUT http://localhost:8080/api/v1/config/ip_allowlist \  
-H "Content-Type: application/json" \  
-d '{"config_value":"127.0.0.1,0:0:0:0:0:0:0:1,192.168.65.1"}'
```

9. Cấu hình sau cài đặt (không cần redeploy)

Điểm mạnh của hệ thống: hầu hết cấu hình nghiệp vụ nằm trong bảng `config` và `mapping_rule` của DB, sửa qua UI (trang **Config**) hoặc API, **đọc lại theo từng request** nên có hiệu lực ngay, không cần build/khởi động lại.

9.1. Các key config đã seed

config_key	Giá trị seed	Ý nghĩa
abort_threshold	0.2	Tỉ lệ record bị loại > ngưỡng này → huỷ cả batch (FAILED)
ip_allowlist	127.0.0.1,0:0:0:0:0:0:1	Danh sách IP được phép POST intake
replay_skew_min	5	Cửa sổ chống replay (phút) cho X-Timestamp
xcenter_inbound_path	xcenter-inbound	Thư mục ghi file MNT
filename_pattern	pricesync_<batch_id>_v<version>_<ts>.mnt	Mẫu tên file (phải chứa <code><ts></code> và kết thúc <code>.mnt</code>)

9.2. Sửa config qua API

```
curl http://localhost:8080/api/v1/config # xem tất cả
curl -X PUT http://localhost:8080/api/v1/config/abort_threshold \
-H "Content-Type: application/json" -d '{"config_value":"0.5"}'
```

9.3. Mapping động (đăng ký field mới, không sửa code)

Bảng `mapping_rule` quy định cột nào của file MNT lấy từ field JSON nào. Thêm một dòng qua trang **Mapping** (hoặc `POST /api/v1/mappings`) là hệ thống hiểu và xuất field mới ra cột MNT ngay — không redeploy. Các cột chuẩn (hợp đồng Oracle) được đánh dấu `locked` để không bị đổi/xoá nhầm.

Lưu ý nghiệp vụ: cột MNT là **hợp đồng vị trí** với DataLoader/Xstore. Thêm cột mới chỉ có ý nghĩa khi phía tiêu thụ (Xstore) đã mong đợi cột đó.

10. Xử lý sự cố thường gặp

Triệu chứng	Nguyên nhân & cách xử lý
<code>Could not resolve placeholder 'HQ_API_KEY'</code> (app chết khi khởi động)	Thiếu biến môi trường bắt buộc. Chạy kèm <code>HQ_API_KEY=...</code> <code>HQ_HMAC_SECRET=...</code> (fail-fast là chủ ý — xem §4).
POST trả 403 Forbidden	IP nguồn không nằm trong <code>ip_allowlist</code> . Trên Docker thêm <code>192.168.65.1</code> (§8.4). Xem log <code>WARN</code> in ra IP bị chặn để biết cần thêm IP nào.
POST trả 401 Unauthorized	Sai/thiếu <code>X-API-Key</code> , sai chữ ký HMAC (body lúc ký ≠ lúc gửi), hoặc <code>X-Timestamp</code> lệch quá cửa sổ (mặc định ±5 phút). Đảm bảo ký đúng <code>ts+body</code> và đồng hồ máy chuẩn.
<code>Connection refused</code> / <code>FATAL: database "price_sync" does not exist</code>	Postgres chưa chạy hoặc chưa tạo DB / sai user. Xem §6.1: <code>brew services start postgresql@16</code> + <code>createdb price_sync</code> , hoặc override <code>SPRING_DATASOURCE_*</code> .
Flyway checksum mismatch trên DB local cũ	Một migration đã áp dụng bị sửa nội dung. Cách chắc chắn trên DB dev là reset sạch để Flyway chạy lại từ đầu: <code>dropdb price_sync && createdb price_sync</code> (cũng xoá dữ liệu test cũ). DB trong Docker luôn mới nên không dính. (Project chưa cài flyway-maven-plugin nên không có lệnh <code>flyway:repair</code> .)
Cổng bận: <code>Port 8080/5173/5432 already in use</code>	Có tiến trình khác chiếm cổng. Tắt nó (<code>lsof -i :8080</code> → <code>kill</code>) hoặc đổi cổng (Spring: <code>server.port</code> ; Vite: <code>--port</code> ; sửa mapping cổng trong compose).
Sửa code Java nhưng "không thấy đổi"	JVM đang chạy bản cũ. Sửa <code>.java</code> phải khởi động lại backend . Chỉ config trong DB mới có hiệu lực không cần restart.
Không nhận được email alert khi batch FAILED	Bình thường nếu chưa có SMTP ở <code>MAIL_HOST</code> — gửi mail là "best-effort", lỗi chỉ log. Muốn xem: dùng Mailpit (Docker, §5) tại <code>:8025</code> , hoặc đặt <code>MAIL_*</code> trở SMTP thật.

11. Phụ lục

11.1. Cổng dùng đến

Cổng	Dịch vụ	Khi nào
8080	Spring Boot (API + UI production)	mọi chế độ
5173	Vite dev server (UI hot-reload)	chỉ local dev
5432	PostgreSQL	local dev (Docker không mở ra host)
8025	Mailpit — hộp thư web	chỉ Docker
1025	Mailpit — SMTP nội bộ	chỉ Docker

11.2. Endpoint chính

Method & path	Bảo mật	Mô tả
POST /api/v1/price-events	4 lớp (IP+HMAC+key+ts)	HQ gửi lô giá → 202
POST /api/v1/events/{id}/retry	không	Operator re-drive batch FAILED
GET /api/v1/events	không	Danh sách batch
GET /api/v1/events/{id}	không	Chi tiết batch + records
GET /api/v1/events/{id}/logs	không	Timeline vòng đời batch
GET /api/v1/events/metrics	không	Đếm batch theo trạng thái
GET-PUT /api/v1/config/{key}	không	Đọc/sửa config trong DB
GET-POST-DELETE /api/v1/mappings	không	Quản lý luật mapping

11.3. Migration schema (Flyway tự chạy)

Toàn bộ nằm ở `backend/src/main/resources/db/migration/`, chạy theo thứ tự V1 → V19 khi app khởi động. Không cần chạy tay.

Migration	Nội dung chính
V1–V2	Bảng <code>price_batch</code> , <code>price_record</code> (+ unique constraint idempotency)
V3–V5	Cột lease (claim/reaper), validation record, retry/backoff
V6	Bảng <code>batch_log</code> (audit vòng đời)
V7–V13	Bảng <code>config</code> + seed connection/threshold; cột <code>extras</code> (JSONB)
V14–V16, V19	Bảng <code>mapping_rule</code> + seed cột chuẩn + validate động + cờ <code>locked</code>
V18	Cột <code>output_file</code> trên <code>price_batch</code>

11.4. Lệnh hay dùng

<code>docker compose up --build</code>	Dựng & chạy toàn hệ thống (Cách A)
<code>./mvnw spring-boot:run</code>	Chạy backend (trong <code>backend/</code> , kèm env)
<code>./mvnw test</code>	Chạy toàn bộ test
<code>./mvnw clean package -DskipTests</code>	Đóng jar production
<code>npm run dev</code>	Chạy UI hot-reload (trong <code>frontend/</code>)
<code>npm run build</code>	Build UI vào static/ của Spring

Price Sync — Hướng dẫn cài đặt · Tài liệu v1.0 · 23/07/2026 · Sinh tự động từ mã nguồn repo XuanSang2005/Price_Sync. Không chứa secret thật; mọi giá trị API key / HMAC / mật khẩu là placeholder cho dev — thay bằng giá trị an toàn ở production.